



Seguridad de los Datos en la Nube

Por qué la información del piso de máquinas está protegida

Casino Atlántico · Manatí, Puerto Rico

10 de julio de 2026

Informe basado en una auditoría de seguridad de 10 revisiones independientes con verificación cruzada, y en las 12 medidas de refuerzo aplicadas al sistema.

1. Resumen ejecutivo

La aplicación del piso de máquinas del Casino Atlántico Manatí guarda su información en la nube (Supabase, una plataforma de base de datos sobre PostgreSQL). Que los datos estén "en la nube" no significa que estén expuestos: significa que viven en un centro de datos profesional, cifrados, y que solo se puede llegar a ellos pasando por varias capas de verificación que nosotros controlamos.

Para comprobarlo, se ejecutó una auditoría con 10 revisiones independientes de seguridad — cada una examinó una dimensión distinta del sistema (permisos de base de datos, manejo de contraseñas y sesiones, cifrado del transporte, dependencias de software, el proceso de despliegue, inyección de código, y escalada de privilegios). Un proceso de consenso cruzó los 10 informes, descartó falsas alarmas y produjo un plan de 12 medidas. Las 12 están aplicadas.

Conclusión: ninguna persona sin cuenta autorizada puede leer un solo dato; ninguna cuenta sin rol de administrador puede modificar nada; los datos viajan siempre cifrados; y las llaves y secretos del sistema nunca están en el código ni en el dispositivo sin cifrar.

2. Qué información se protege

- **Datos operativos del casino:**
285 máquinas con su ubicación, juego, fabricante y apuestas; unas 25,650 filas de ingresos diarios (coin-in / win) por máquina; y el historial de compras, reubicaciones y cambios de juego.
- **Datos de las personas usuarias:**
correo, nombre y rol (admin o viewer) de cada empleado con acceso, más su contraseña (que nunca se guarda como texto: solo un hash criptográfico).
- **Secretos del sistema:**
las llaves que administran la base de datos, que nunca aparecen en el código fuente ni en la aplicación instalada.

3. Las seis capas de defensa

La seguridad no depende de una sola barrera sino de capas independientes: para llegar a un dato, un atacante tendría que vencerlas todas, y cada capa está pensada para que el fallo de otra no la arrastre.

Capa 1 — El dato viaja cifrado (transporte)

Toda comunicación entre la app y la nube va por HTTPS/TLS: cifrada de extremo a extremo, imposible de leer o alterar en tránsito (WiFi público incluido). El código además se niega a arrancar si alguien configurara por error una dirección sin cifrado, y el servidor web exige HTTPS permanentemente (HSTS).

Capa 2 — Nadie entra sin identificarse (autenticación)

Solo se accede con correo y contraseña verificados por Supabase Auth. Las contraseñas se guardan con hash bcrypt — ni el propio proveedor puede leerlas. Cada sesión usa tokens JWT firmados y de vida corta que se renuevan automáticamente.

Capa 3 — La sesión se guarda cifrada en el dispositivo

El token de sesión ya no se guarda en texto plano: en iOS y Android vive en el Keychain/Keystore del sistema operativo — el mismo almacén cifrado por hardware donde el teléfono guarda sus propias credenciales. Si alguien pierde el dispositivo o le extraen un respaldo, el token no es legible. Al cerrar sesión, el estado local se borra siempre, incluso si falla la red — importante en equipos compartidos del piso.

Capa 4 — El servidor decide quién ve qué (autorización RLS)

Esta es la capa más importante. La base de datos aplica Row-Level Security: reglas dentro del propio motor PostgreSQL que se evalúan en cada consulta, sin importar desde dónde venga.

- Un visitante sin sesión recibe cero filas. Siempre.
- Una cuenta autenticada solo lee datos si además tiene un rol reconocido (admin o viewer) otorgado explícitamente — una cuenta recién creada no ve nada hasta que se le concede.
- Solo el rol admin puede escribir o modificar. Un viewer que intente editar directamente contra el servidor (saltándose la app) es rechazado por la base de datos.
- Nadie puede ascenderse a sí mismo a administrador: una regla explícita (WITH CHECK) fija el rol de cada perfil; cambiarlo requiere la llave de servicio, que nunca sale del entorno administrativo.

Por esto la llave pública ("anon key") que la app lleva incorporada no es un riesgo: no abre ningún dato por sí sola; solo permite tocar la puerta. Quien decide es siempre el servidor.

Capa 5 — El navegador está blindado (web)

La versión web se sirve con cabeceras de seguridad estrictas: una Content-Security-Policy que solo permite conectar con nuestros propios servidores y Supabase (un script malicioso no tendría a dónde exfiltrar datos), prohibición de incrustar la app en páginas ajenas (anti-clickjacking), y bloqueo de rastreo de origen. Además las fuentes tipográficas ahora se sirven desde nuestro propio dominio: la app en producción no hace ni una sola petición a terceros ajenos a Supabase.

Capa 6 — La cadena de suministro está controlada

Las llaves administrativas viven solo como secretos cifrados de GitHub, en un entorno protegido y con procesos de mínimo privilegio; las herramientas del pipeline están fijadas a versiones exactas e inmutables (commit SHA) para que nadie pueda sustituirlas; y las librerías con avisos de seguridad fueron actualizadas a versiones parchadas. El repositorio ignora todo archivo de credenciales (.env*) para que un descuido no las publique jamás.

4. Qué encontró la auditoría y qué se hizo

Riesgo detectado (consenso de 10 auditores)	Nivel	Medida aplicada
Cualquier cuenta autenticada podía leer todo el dataset financiero	ALTO	Lecturas restringidas por rol explícito en RLS (migración 0008) + desactivar registro abierto en Supabase
Token de sesión guardado sin cifrar en el dispositivo	ALTO	Keychain/Keystore cifrado vía expo-secure-store en iOS/Android; CSP y vidas cortas de token en web
Web sin cabeceras de seguridad (CSP, HSTS, anti-clickjacking)	MEDIO	Bloque completo de cabeceras en vercel.json
Pipeline CI con privilegios amplios y secretos sin protección	MEDIO	Mínimo privilegio, entorno protegido "production", acciones fijadas a SHA, npm sin scripts arbitrarios
Dependencias con avisos de seguridad conocidos	MEDIO	Overrides a versiones parchadas (dompurify, form-data, undici); upgrade Expo 57 planificado
Auto-promoción a admin bloqueada solo implícitamente	BAJO	Regla explícita WITH CHECK que fija el rol del perfil
Cierre de sesión podía fallar y dejar la sesión activa	BAJO	Limpieza local garantizada (finally) aunque falle la red
Datos en caché visibles tras cerrar sesión en equipo compartido	BAJO	El almacén de datos se purga al cerrar sesión; nada se descarga sin sesión activa
Ediciones bloqueadas por el servidor parecían guardarse	BAJO	Cada escritura verifica las filas realmente afectadas y revierte si el servidor la negó

Riesgo detectado (consenso de 10 auditores)	Nivel	Medida aplicada
Credenciales podían colarse al repositorio por descuido	BAJO	.gitignore cubre todo .env*; llaves reales jamás en el código
Sin verificación de que la conexión usara HTTPS	BAJO	La app se niega a arrancar con una URL sin cifrar
Fuentes servidas por un CDN de terceros (Google)	BAJO	Fuentes auto-hospedadas; cero peticiones a terceros en producción

5. ¿Por qué la nube es más segura que un archivo local?

Una duda razonable: "¿no estaría más seguro esto en una computadora del casino?" En la práctica, no:

- **Cifrado en reposo:**
Supabase cifra los discos donde viven los datos (AES-256) y opera en centros de datos con certificación SOC 2, con redundancia y respaldos — un Excel en una laptop no tiene nada de eso, y se va con la laptop si la roban.
- **Control de acceso central:**
en la nube, dar de baja a un empleado es revocar una cuenta y su acceso muere en el instante, en todos los dispositivos. Con archivos locales, cada copia repartida es una fuga imposible de revocar.
- **El servidor manda:**
las reglas RLS viven dentro del motor de la base de datos. Aunque roben un teléfono con la app instalada, el servidor sigue exigiendo sesión válida y rol correcto para cada fila.
- **Auditoría y parches:**
la plataforma se parcha continuamente contra vulnerabilidades nuevas, algo que una PC de oficina rara vez recibe a tiempo.

6. Lo que la tecnología no puede hacer sola

Estas medidas operativas quedan del lado humano; están recomendadas por la auditoría y se configuran en los paneles de Supabase y GitHub (no en el código):

- Aplicar la migración 0008 en Supabase y desactivar el registro abierto de cuentas (self-signup) — juntos cierran el hallazgo de mayor riesgo.
- Activar CAPTCHA y límites de intentos de acceso (anti fuerza bruta), y la protección contra contraseñas filtradas.
- Activar rotación de refresh-tokens y vidas cortas de sesión.
- Restringir CORS al dominio oficial de la app.
- Activar el escaneo de secretos de GitHub y proteger el entorno "production" con revisores obligatorios.
- Y lo de siempre: contraseñas fuertes y únicas, no compartir cuentas, y retirar accesos al momento en que alguien deja el equipo.

7. Conclusión

Tras la auditoría de 10 revisiones independientes y la aplicación de las 12 medidas del plan de consenso, la información del Casino Atlántico Manatí está protegida por seis capas independientes: cifrado en tránsito, autenticación verificada, sesión cifrada en el dispositivo, autorización fila por fila dentro de la base de datos, navegador blindado y cadena de suministro controlada.

Estar en la nube no es el riesgo — es parte de la defensa: los datos viven cifrados en infraestructura profesional y auditada, y cada acceso, venga de donde venga, tiene que demostrar quién es y qué rol tiene antes de ver una sola fila.

Documento generado el 10 de julio de 2026. Evidencia técnica: migraciones supabase/migrations/0001–0008, src/lib/supabase.ts, src/lib/authStorage.ts, vercel.json, .github/workflows/ y el informe de consenso de la auditoría (PR #8).